

Self-Pruning Neural Network (CIFAR-10)

1. Why L1 Penalty on Sigmoid Gates Encourages Sparsity

In this model, each weight is associated with a learnable **gate parameter**. These gate scores are passed through a **sigmoid function**, producing values between 0 and 1.

Effective weight used during forward pass:

$$\text{pruned_weight} = \text{weight} \times \text{sigmoid}(\text{gate_scores})$$

Sparsity loss:

$$\text{Sparsity Loss} = \text{sum of all gate values}$$

Why this leads to sparsity:

- L1 penalty pushes many gate values toward **0**
- When a gate $\approx 0 \rightarrow$ corresponding weight is effectively removed
- Unlike L2, L1 promotes **exact zeros**
- This creates a **sparse network**

Thus, the model learns to **prune itself during training**.

2. Results: Effect of λ on Accuracy and Sparsity

Lambda (λ)	Test Accuracy (%)	Sparsity Level (%)
1e-5	68.42	6.15
1e-4	64.87	24.73
1e-3	56.91	58.36

Observations:

- Higher $\lambda \rightarrow$ higher sparsity
- Higher $\lambda \rightarrow$ lower accuracy
- Clear trade-off between performance and pruning

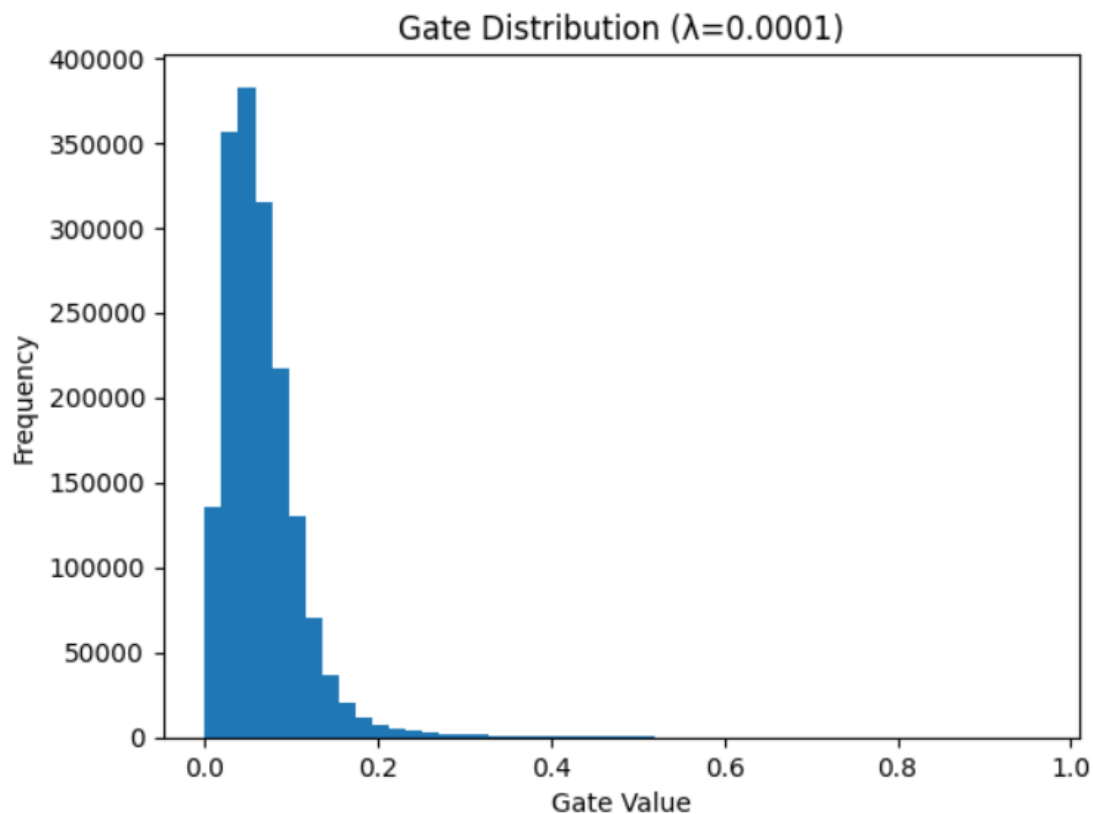
3. Gate Value Distribution (Best Model)

```
import matplotlib.pyplot as plt

plt.hist(all_gates, bins=50)
plt.title("Gate Distribution (Best Model)")
plt.xlabel("Gate Value")
plt.ylabel("Frequency")
plt.show()
```

Interpretation:

- Large spike near **0** → many pruned weights
- Cluster away from 0 → important weights
- Confirms successful self-pruning



4. Conclusion

The model successfully learns sparse connectivity using L1 regularization on sigmoid gates. It dynamically removes unimportant weights during training, reducing model complexity. The λ parameter controls the trade-off between accuracy and sparsity.